

# Weeks 2-10

## Grades 2-3

Small counts and concrete arguments; adult reading help is welcome.

Grade bands are entry points. Begin with objects, then try to explain what happens.

Give one page at a time. The packet is a menu, not a checklist for one hour.

Keep facilitator solutions separate from student pages.

| Week and investigation           | PDF pages |
|----------------------------------|-----------|
| 02 Switches and lamps            | 2-3       |
| 03 Repeating secret machines     | 4-5       |
| 04 Around the ring               | 6-7       |
| 05 Tower cities                  | 8-9       |
| 06 Code-breaking with guarantees | 10-11     |
| 07 Take-away games               | 12-13     |
| 08 Rooks and piles               | 14-15     |
| 09 Bouncing paths                | 16-17     |
| 10 Bridges and delivery routes   | 18-19     |

Print US Letter, single-sided, at 100% / Actual Size.

Page ranges above count this cover as page 1. Bookmarks open each week.

Weekly footers retain their own page numbers and activity IDs for the use log.

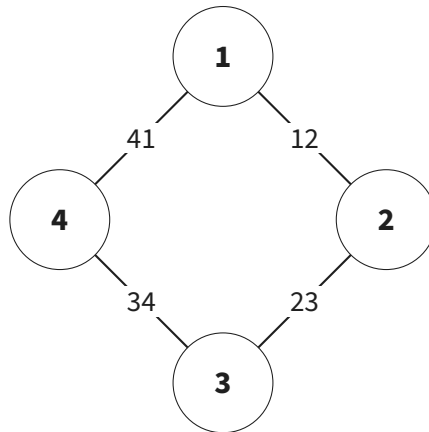
Revised September 20, 2026. Planned activities; record actual use after teaching.

# Which pictures are possible?

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Start all lamps OFF. Press a line to flip **both** end lamps. Dark means ON. A lamp already ON turns OFF. Write 12 for the line joining lamps 1 and 2.



## 1. Three targets.

Reset to all OFF before each target. Find moves, or give a reason the target cannot happen.

| ON lamps | My moves or my reason |
|----------|-----------------------|
| 1 and 2  |                       |
| 1 and 3  |                       |
| 1 only   |                       |

## 2. Look at one move closely.

The two end lamps might both be OFF, both ON, or one of each. How does the number of ON lamps change in these three cases?

**A rule that survives every move:**

Try it with objects. Explain by pointing, talking, or drawing.

# Find the whole world

## 3. Make a complete collection.

Draw every reachable picture in the boxes: a dot for each lamp, filled if ON. Keep the order 1, 2, 3, 4. The all-OFF picture counts. There may be more boxes than you need.

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

**Our count:** \_\_\_\_\_ How do you know none are missing?

## 4. Make two far-apart lamps.

Choose any two lamps. Trace a path between them. Press each line on your path once. What happens to the lamps in the middle?

**Explain why a path changes only its two ends.**

|  |
|--|
|  |
|--|

**Go further:** What if the ring has 5 lamps? Can every picture with an even number of ON lamps still be made? Pair its ON lamps and try paths.

Try it with objects. Explain by pointing, talking, or drawing.

# A code you can undo

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Use cards A, B, C, D. A key uses every letter exactly once in its output row. One turn replaces **every** letter at once. Keep spaces and letter positions fixed.

| Input | A | B | C | D |
|-------|---|---|---|---|
| Key P | B | C | A | D |
| Key Q | B | A | D | C |

## 1. What stays the same?

Encode ABBA with P. Could any valid key turn ABBA into ABCD? Explain. Then decode P's output CAAC.

**My messages and a reason:**

## 2. Keep coding the code.

Start with ABCD. Apply P again and again until **all four letters** are back in their starting places. Do the same with Q. Count the first positive return.

| Turn | Key P | Key Q |
|------|-------|-------|
| 0    | ABCD  | ABCD  |
| 1    |       |       |
| 2    |       |       |
| 3    |       |       |
| 4    |       |       |

**Look closer:** Follow just A through P. What happens to D? Is one letter returning enough to say the *whole machine* has reset?

Try it with objects. Explain by pointing, talking, or drawing.

# Build a slow machine

## 3. Make a key that takes four turns.

Write each of A, B, C, D exactly once under the line. Test it with the whole message ABCD. Draw arrows to show the route of each letter.

|        |   |   |   |   |
|--------|---|---|---|---|
| Input  | A | B | C | D |
| My key |   |   |   |   |

### Arrow loops and my test:

## 4. Can four letters take six turns?

Try it. Then sort your keys by their loops. A letter that stays put is a loop of size 1. What are all the ways to split four letters into loops?

| Loop sizes | First turn when all are home |
|------------|------------------------------|
|            |                              |
|            |                              |
|            |                              |
|            |                              |
|            |                              |
|            |                              |
|            |                              |
|            |                              |
|            |                              |

**Go further:** Add a fifth letter. Can you build a machine that really does take six turns? Try two separate loops.

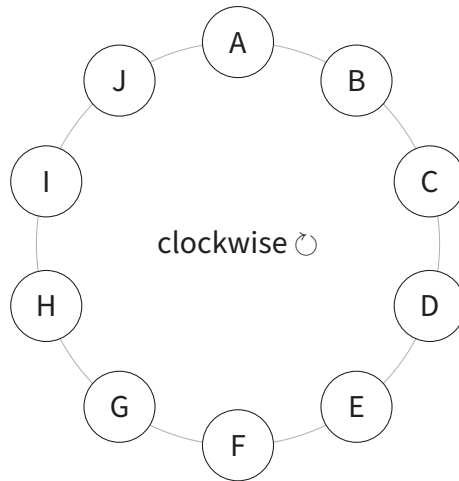
Try it with objects. Explain by pointing, talking, or drawing.

# One clue controls the whole code

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Our small alphabet is A, B, C, D, E, F, G, H, I, J. To encode, move the **same number of places clockwise** for every letter. After J comes A.



## 1. Recover the key.

You learn that message A becomes code D. Decode **GDG**. Then encode **BAG**. Check by reversing your moves.

**My shift, decoded message, and new code:**

## 2. A clue that cannot fit.

Someone also claims B becomes F under that same key. Could both clues be true? Show why on the ring.

**My reason:**

Try it with objects. Explain by pointing, talking, or drawing.

# A code that skips places

## 3. Repeat a two-step code.

Start at A. Take two clockwise steps each turn. Write each **landing** until you first return to A. Do not include skipped-over letters. Which letters never appear?

**My two-step route and the missing letters:**

## 4. Repeat a three-step code.

Start at A again. Take three steps each turn. Does this rule visit every letter before returning? Give the whole route so someone else can check.

**My three-step route:**

## 5. Explain the split.

Start the two-step rule at B instead. How do its landings relate to your route from A? Why can the A-route never jump into the B-route?

**Two loops, and why they cannot meet:**

**Go further:** Which step sizes from 1 to 9 visit all ten letters? Can a two-step code still be *decoded* even though one repeated route skips letters?

Try it with objects. Explain by pointing, talking, or drawing.

# One clue, two cities?

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Use heights 1, 2, 3 once in each row and once in each column. A number outside counts towers seen from that end; taller towers hide shorter ones.

## 1. Build a city.

First make any legal city with your towers. Then add the clue shown here: three towers must be visible down the first column.

3

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

3

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

## 2. Find every city with that clue.

Record two different cities above. Are there more? Keep the board in the same orientation.

**Explain why your list is complete. What can go in the top row?**

**Hint after trying:** The first column is forced. Try each possible order of the two remaining heights in the top row.

Build first. Explain by pointing, talking, or drawing.

# How little information is enough?

Name: \_\_\_\_\_

Date: \_\_\_\_\_

## 3. Make just one city possible.

Keep the top clue 3. Add a right-side clue of 2 to the **middle row**. Which of your two cities survives? Show why.

3

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

2

## 4. Is either clue unnecessary?

Cover the right clue: your first page gives two answers. Now cover only the top clue. Find two cities that keep the right clue.

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

2

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

2

**Would any single outside clue ever force an entire 3-by-3 city?**

**Try a reason covering all clues:** Keep the row or column that a clue sees fixed. Can you swap two other rows or columns?

Build first. Explain by pointing, talking, or drawing.

# A method that works for every secret

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Two colors: R and B. Repeats allowed. A test earns one point for each color in the correct position. You hear **only the total**, not which positions match.

## 1. Explore three places.

Play a round without a test limit. Keep the old tests and scores visible. Then try this investigation.

## 2. Change exactly one place.

Compare RRR with BRR. Can the score stay the same? What does an increase or a decrease tell you?

| Test | My code | Score | Still possible |
|------|---------|-------|----------------|
| 1    |         |       |                |
| 2    |         |       |                |
| 3    |         |       |                |
| 4    |         |       |                |

## 3. Three tests, then name the secret.

Try RRR, BRR, and RBR. How can these reveal all three places? Check your method on a partner’s secret.

### Why my method always determines the third place too:

**Counting rule:** Naming a known secret costs no test here. In the original game, submitting it as a winning guess may cost one more turn.

Build first. Explain by pointing, talking, or drawing.

# Can anyone always do it in two?

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Two colors: R and B. Repeats allowed. A test earns one point for each color in the correct position. You hear **only the total**, not which positions match.

## 4. The difficult first answer.

Suppose RRR scores 2. Exactly these secrets remain:

BRR      RBR      RRB

## 5. Can a last test separate all three?

Try rows of the table and circle repeated scores. What is the only feature that changes among the three secrets? Explain why a test can tell only whether the secret's B is in one of its B positions. **Stop calculating rows once you can explain why at most two scores occur.**

| Test | BRR | RBR | RRB |
|------|-----|-----|-----|
| RRR  |     |     |     |
| RRB  |     |     |     |
| RBR  |     |     |     |
| RBB  |     |     |     |
| BRR  |     |     |     |
| BRB  |     |     |     |
| BBR  |     |     |     |
| BBB  |     |     |     |

## 6. Does a different first test help?

Try BRB. In *both* a secret and every test, swap R/B at places 1 and 3. The first test becomes RRR. Check that each matching place still matches. Do this at every B position of any first test. Why are all scores unchanged? Discuss with an adult.

**Why no first test can guarantee success in only two tests:**

**What we proved:** Task 5 rules out two tests starting with RRR. With task 6, it covers every first test; together with page 1, three is optimal.

Build first. Explain by pointing, talking, or drawing.

# The missing move changes everything

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Two players alternate. Take **1, 3, or 4** counters each turn. Taking 2 is illegal. Never take more than remain. Taking the last counter wins.

## 1. Play and suspect.

Try piles of 5, 6, and 7. Trade who goes first. A win gives you a clue; it does not yet prove a strategy.

## 2. Solve small positions.

L means the player about to move loses against perfect replies. W means that player has a winning strategy. Zero is L: the previous player took the last counter.

|   |   |   |    |    |    |    |
|---|---|---|----|----|----|----|
| 0 | 1 | 2 | 3  | 4  | 5  | 6  |
|   |   |   |    |    |    |    |
| 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|   |   |   |    |    |    |    |

## 3. Justify each mark.

To label W, draw one legal move to an L. To label L, check **every** legal move and show it leads to a W. Start with 0 and work upward.

**One W with its winning move; one L with all its replies:**

Build first. Explain by pointing, talking, or drawing.

# One good move, or every bad move?

Name: \_\_\_\_\_

Date: \_\_\_\_\_

Two players alternate. Take **1, 3, or 4** counters each turn. Taking 2 is illegal. Never take more than remain. Taking the last counter wins.

## 4. Two claims to test.

Alex says: "From 8, take 3 because taking more is better." Bea says: "From 10, there is only one winning move." Use your chart to check both.

**Alex's move and a better move, with reasons:**

**Every winning move from 10:**

## 5. Make a strategy card.

Choose a pile between 8 and 14. If it is W, show a winning first move and how you will use your chart after that. If it is L, explain why every first move gives away the advantage.

**My pile and my strategy certificate:**

**Go further:** Mark 14 through 20. Do you see a repeating pattern? What would you need to prove before trusting it for a pile of 100?

Build first. Explain by pointing, talking, or drawing.

# Map the traps

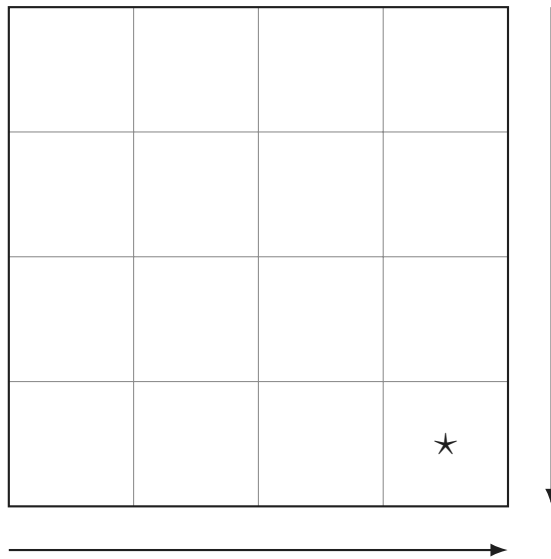
Name: \_\_\_\_\_

Date: \_\_\_\_\_

Share one token. Move right OR down any positive number of squares. No diagonal moves. Reach the star to win.

## 1. Play, then make a map.

Try different starts. Mark a square T if the player who receives it must lose against perfect replies. Mark W if that player can force a win. Use ? while unsure.



Treat the star as already finished: the next player has no move. Start your map near it. A single lost game does not prove T!

## 2. A claim that survives every reply.

Choose one T. Show that **every** move from it gives the next player a W. Then choose one W and show **one** move to a T.

**My two squares and reasons:**

Try it with objects. Explain by pointing, talking, or drawing.

# The board is two piles

Name: \_\_\_\_\_

Date: \_\_\_\_\_

## 3. Count what is left.

Put the token 3 squares left and 2 squares above the star. Make a pile of 3 and a pile of 2. A right move shrinks the first pile; a down move shrinks the second.

Pile rule: remove any positive number from ONE pile. Take the last counter to win. Play both versions and match their moves.

## 4. Prove a rule for every rectangle.

Which two-pile positions are traps? Explain these two things with counters:

1. Every move from a trap breaks your pattern.
2. From any other position, you can make the pattern.

**My pattern and why both statements hold:**

## 5. Add a third pile.

Keep the same pile rule. Try  $(1, 1, 1)$ , then  $(1, 2, 3)$ . Does “two matching piles” still guarantee a trap? List all first moves from  $(1, 2, 3)$  and find a winning reply to each.

**A surprise or a new conjecture:**

**Make a puzzle:** Choose a two-pile start with exactly one winning move. Let a partner test it.

Try it with objects. Explain by pointing, talking, or drawing.

# Unbounce the ball

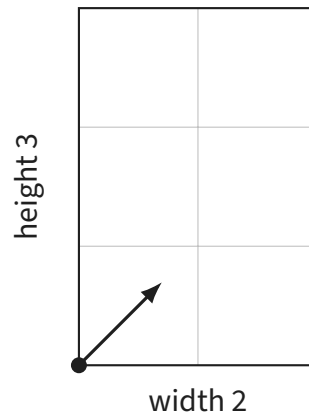
Name: \_\_\_\_\_

Date: \_\_\_\_\_

Start at the bottom-left dot, at a 45-degree angle. At a side wall reverse left/right; at a top or bottom wall reverse up/down. Stop at the FIRST corner.

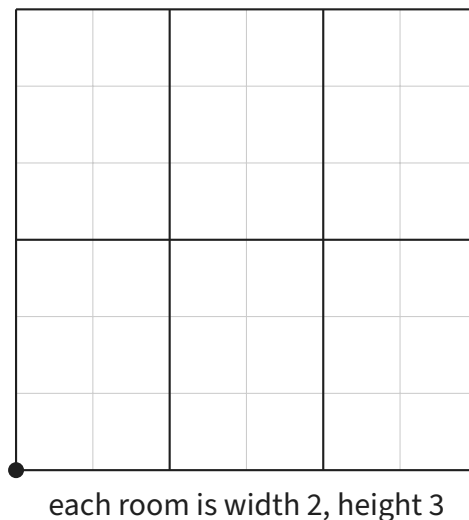
## 1. Find a route.

Trace this table. Mark each bounce and the final corner.



## 2. Instead of bouncing, copy the room.

Below are mirror copies of that table. Continue the path as one straight diagonal. Mark the *first* point where it meets both a thick vertical and a thick horizontal wall.



**Explain with a fold or your hands:** Why does crossing into a mirror room stand for a bounce in the original room?

Try it with objects. Explain by pointing, talking, or drawing.

# When do two walls meet?

Name: \_\_\_\_\_

Date: \_\_\_\_\_

## 3. Count in two rhythms.

For a width-2, height-3 table, thick vertical walls occur after 2, 4, 6, ... horizontal units. Thick horizontal walls occur after 3, 6, 9, ... vertical units. On a 45-degree path, those distances are equal.

What is their first shared distance? Why would an earlier number fail?

## 4. Predict before drawing.

Use copied rooms, lists of multiples, or a sketch. Count the number of room widths and heights to that first meeting.

| Width | Height | First shared | Widths | Heights |
|-------|--------|--------------|--------|---------|
| 2     | 3      |              |        |         |
| 3     | 6      |              |        |         |
| 3     | 9      |              |        |         |
| 4     | 6      |              |        |         |

**Fold back mentally:** Crossing one whole room width takes you to the right, two to the left, three to the right. What do room heights tell you? Predict the ending corner in each row.

### A prediction and a reason:

## 5. Design a challenge.

Make a rectangle that ends at the top-right *after at least one bounce*. Then make a table ending at the top-left. Give your partner the sizes and keep your explanation hidden.

### My two sizes, corners, and checks:

Try it with objects. Explain by pointing, talking, or drawing.

# A map you can draw in one walk

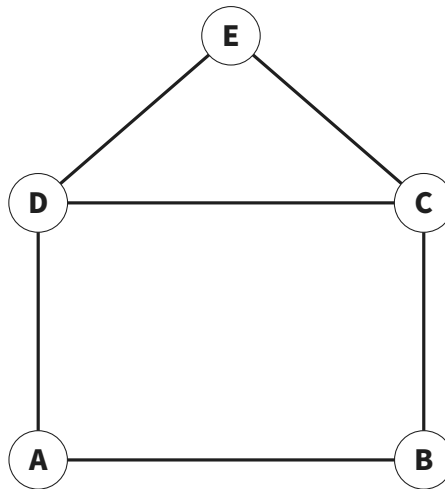
Name: \_\_\_\_\_

Date: \_\_\_\_\_

Travel every line exactly once without jumping. Visiting a dot more than once is allowed. You choose the start. Mark used lines with counters, or number them in walking order.

## 1. Find a route through the house.

Can you finish where you started? If not, find a walk with different starting and ending dots.



## 2. Pair an arrival with a departure.

At each dot, count the lines meeting there. This count is its **degree**. Circle the dots of odd degree.

**Why must the start and finish be special?**

**Make a conjecture:** How many odd dots can a one-walk map have? What changes if it must return to the start?

Try it with objects. Explain by pointing, talking, or drawing.

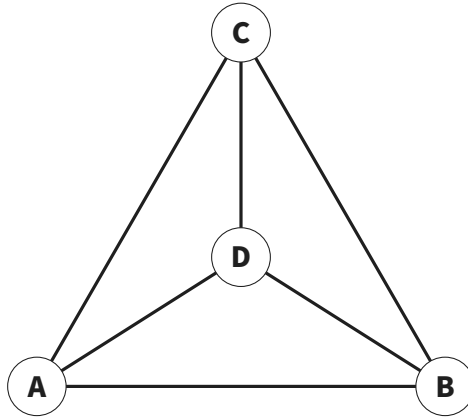
# Repair an impossible map

Name: \_\_\_\_\_

Date: \_\_\_\_\_

## 3. One island inside three others.

Try to walk every line once. There are six lines and four dots. Does changing the start help?



**An argument covering every possible start:**

## 4. Add one bridge.

Draw a second bridge joining two dots already connected. Draw it as a curved line so both bridges are visible. Can the new map be walked using each bridge once? Choose the start carefully.

**Which bridge did I add? My complete route:**

## 5. Build a puzzle for a partner.

Draw a connected map with exactly two odd dots. Every bridge must join two different dots. Keep a complete one-walk solution hidden. Ask your partner where a route must start. If your map fails, revise it and explain what changed.

Try it with objects. Explain by pointing, talking, or drawing.